

MIENTRAS

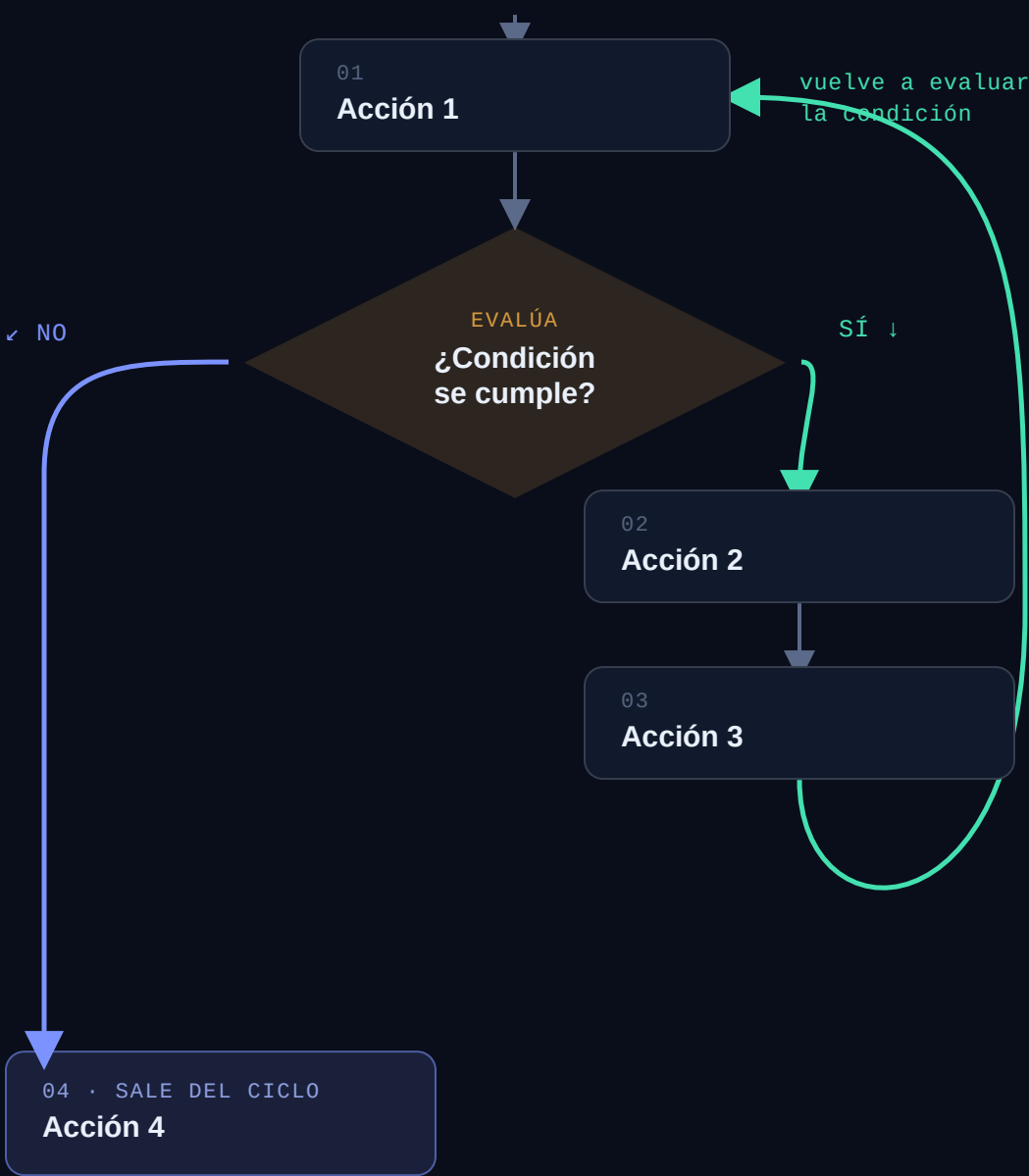
Mientras (condición) ... FinMientras

Es una estructura repetitiva con la condición al inicio del bucle: mientras se cumpla, el bloque se repite una y otra vez; en cuanto deja de cumplirse, el ciclo termina.

- evalúa la condición antes de entrar
- puede ejecutarse 0 veces
- termina cuando la condición es falsa

>_ cómo se ve el ciclo

Diagrama de flujo



- **Acción 1 corre siempre**

Se ejecuta antes de preguntar nada — prepara lo que la condición necesita evaluar (por ejemplo, leer un dato).
- **La condición se evalúa primero**

Aquí la pregunta va al inicio del ciclo — por eso MIENTRAS puede no ejecutar el bloque interno ni una sola vez.
- **Si es verdadera → entra al bloque**

Corre Acción 2 y Acción 3, y luego vuelve a evaluar la condición desde el principio.
- **Si es falsa → sale**

El ciclo se rompe de inmediato y el programa continúa en Acción 4.

>_ diff --repetir mientras

MIENTRAS vs. REPETIR

MIENTRAS (WHILE)

- Condición **al inicio** del bucle
- Puede ejecutarse **0 veces**
- Entra al bloque solo si la condición es **verdadera**

REPETIR (posterior)

- Condición **al final** del bucle
- Se ejecuta **al menos 1 vez**
- Sale del bloque cuando la condición es **verdadera**

>_ sintaxis y ejemplo

De la teoría al código

seudocódigo – patrón general

SINTAXIS

```
1 Mientras (condición)
2     .....
3     .....
4 FinMientras
```

Las líneas entre **Mientras** y **FinMientras** son el bloque que se repite mientras la condición sea verdadera.

ejemplo práctico – contar hasta 3

EN VIVO

```
1 Inicio
2   Entero num
3   num = 0
4   Mientras (num<3)
5       num = num+1
6       Escribir "El número es " num
7   FinMientras
8 Fin
```

Como **num** empieza en 0, la condición ya es verdadera desde el primer chequeo: el bloque corre **3 veces** (1, 2 y 3) y se detiene.



Ciclo de pre-condición

MIENTRAS evalúa la condición al inicio del bloque, no al final. Por eso se le llama ciclo controlado por pre-condición.



Cero ejecuciones posibles

Si la condición es falsa desde el principio, el bloque interno nunca llega a correr.



Úsalo cuando...

No puedes garantizar que el bloque deba ejecutarse: recorrer una lista que podría estar vacía, procesar mientras haya datos disponibles.